

shows empirically that one classifier can have a margin distribution that is uniformly better than that of another classifier, and yet be inferior in test accuracy. On the other hand, Koltchinskii, Panchenko and Lozano [44, 45, 46, 58] have recently proved new margin-theoretic bounds that are tight enough to give useful quantitative predictions.

Attempts (not always successful) to use the insights gleaned from the theory of margins have been made by several authors [9, 37, 50]. In addition, the margin theory points to a strong connection between boosting and the support-vector machines of Vapnik and others [7, 14, 77] which explicitly attempt to maximize the minimum margin.

5 A connection to game theory and linear programming

The behavior of AdaBoost can also be understood in a game-theoretic setting as explored by Freund and Schapire [31, 33] (see also Grove and Schuurmans [37] and Breiman [9]). In classical game theory, it is possible to put any two-person, zero-sum game in the form of a matrix $\mathbf{M}$. To play the game, one player chooses a row i and the other player chooses a column j . The loss to the row player (which is the same as the payoff to the column player) is $\mathbf{M}_{ij}$. More generally, the two sides may play randomly, choosing distributions $\mathbf{P}$ and $\mathbf{Q}$ over rows or columns, respectively. The expected loss then is $\mathbf{P}^T \mathbf{M} \mathbf{Q}$.

Boosting can be viewed as repeated play of a particular game matrix. Assume that the base classifiers are binary, and let $\mathcal{H} = \{h_1, \dots, h_n\}$ be the entire base classifier space (which we assume for now to be finite). For a fixed training set $(x_1, y_1), \dots, (x_m, y_m)$, the game matrix $\mathbf{M}$ has m rows and n columns where

$$\mathbf{M}_{ij} = \begin{cases} 1 & \text{if } h_j(x_i) = y_i \\ 0 & \text{otherwise.} \end{cases}$$

The row player now is the boosting algorithm, and the column player is the base learner. The boosting algorithm's choice of a distribution D_t over training examples becomes a distribution $\mathbf{P}$ over rows of $\mathbf{M}$, while the base learner's choice of a base classifier h_t becomes the choice of a column j of $\mathbf{M}$.

As an example of the connection between boosting and game theory, consider von Neumann's famous minmax theorem which states that

$$\max_{\mathbf{Q}} \min_{\mathbf{P}} \mathbf{P}^T \mathbf{M} \mathbf{Q} = \min_{\mathbf{P}} \max_{\mathbf{Q}} \mathbf{P}^T \mathbf{M} \mathbf{Q}$$

for any matrix $\mathbf{M}$. When applied to the matrix just defined and reinterpreted in the boosting setting, this can be shown to have the following meaning: If, for any

distribution over examples, there exists a base classifier with error at most $1/2 - \gamma$, then there exists a convex combination of base classifiers with a margin of at least 2γ on all training examples. AdaBoost seeks to find such a final classifier with high margin on all examples by combining many base classifiers; so in a sense, the minmax theorem tells us that AdaBoost at least has the potential for success since, given a “good” base learner, there must exist a good combination of base classifiers. Going much further, AdaBoost can be shown to be a special case of a more general algorithm for playing repeated games, or for approximately solving matrix games. This shows that, asymptotically, the distribution over training examples as well as the weights over base classifiers in the final classifier have game-theoretic interpretations as approximate minmax or maxmin strategies.

The problem of solving (finding optimal strategies for) a zero-sum game is well known to be solvable using linear programming. Thus, this formulation of the boosting problem as a game also connects boosting to linear, and more generally convex, programming. This connection has led to new algorithms and insights as explored by Rätsch et al. [62], Grove and Schuurmans [37] and Demiriz, Bennett and Shawe-Taylor [17].

In another direction, Schapire [68] describes and analyzes the generalization of both AdaBoost and Freund’s earlier “boost-by-majority” algorithm [26] to a broader family of repeated games called “drifting games.”

6 Boosting and logistic regression

Classification generally is the problem of predicting the label y of an example x with the intention of minimizing the probability of an incorrect prediction. However, it is often useful to estimate the *probability* of a particular label. Friedman, Hastie and Tibshirani [34] suggested a method for using the output of AdaBoost to make reasonable estimates of such probabilities. Specifically, they suggested using a logistic function, and estimating

$$\Pr_f [y = +1 \mid x] = \frac{e^{f(x)}}{e^{f(x)} + e^{-f(x)}} \quad (7)$$

where, as usual, $f(x)$ is the weighted average of base classifiers produced by AdaBoost (Eq. (3)). The rationale for this choice is the close connection between the log loss (negative log likelihood) of such a model, namely,

$$\sum_i \ln (1 + e^{-2y_i f(x_i)}) \quad (8)$$

and the function that, we have already noted, AdaBoost attempts to minimize:

$$\sum_i e^{-y_i f(x_i)}. \quad (9)$$

Specifically, it can be verified that Eq. (8) is upper bounded by Eq. (9). In addition, if we add the constant $1 - \ln 2$ to Eq. (8) (which does not affect its minimization), then it can be verified that the resulting function and the one in Eq. (9) have identical Taylor expansions around zero up to second order; thus, their behavior near zero is very similar. Finally, it can be shown that, for any distribution over pairs (x, y) , the expectations

$$\mathbb{E} \left[\ln \left(1 + e^{-2yf(x)} \right) \right]$$

and

$$\mathbb{E} \left[e^{-yf(x)} \right]$$

are minimized by the same (unconstrained) function f , namely,

$$f(x) = \frac{1}{2} \ln \left(\frac{\Pr[y = +1 \mid x]}{\Pr[y = -1 \mid x]} \right).$$

Thus, for all these reasons, minimizing Eq. (9), as is done by AdaBoost, can be viewed as a method of approximately minimizing the negative log likelihood given in Eq. (8). Therefore, we may expect Eq. (7) to give a reasonable probability estimate.

Of course, as Friedman, Hastie and Tibshirani point out, rather than minimizing the exponential loss in Eq. (6), we could attempt instead to directly minimize the logistic loss in Eq. (8). To this end, they propose their LogitBoost algorithm. A different, more direct modification of AdaBoost for logistic loss was proposed by Collins, Schapire and Singer [13]. Following up on work by Kivinen and Warmuth [43] and Lafferty [47], they derive this algorithm using a unification of logistic regression and boosting based on Bregman distances. This work further connects boosting to the maximum-entropy literature, particularly the iterative-scaling family of algorithms [15, 16]. They also give unified proofs of convergence to optimality for a family of new and old algorithms, including AdaBoost, for both the exponential loss used by AdaBoost and the logistic loss used for logistic regression. See also the later work of Lebanon and Lafferty [48] who showed that logistic regression and boosting are in fact solving the same constrained optimization problem, except that in boosting, certain normalization constraints have been dropped.

For logistic regression, we attempt to minimize the loss function

$$\sum_i \ln \left(1 + e^{-y_i f(x_i)} \right) \quad (10)$$

which is the same as in Eq. (8) except for an inconsequential change of constants in the exponent. The modification of AdaBoost proposed by Collins, Schapire and Singer to handle this loss function is particularly simple. In AdaBoost, unraveling the definition of D_t given in Fig. 1 shows that $D_t(i)$ is proportional (i.e., equal up to normalization) to

$$\exp(-y_i f_{t-1}(x_i))$$

where we define

$$f_t(x) = \sum_{t'=1}^t \alpha_{t'} h_{t'}(x).$$

To minimize the loss function in Eq. (10), the only necessary modification is to redefine $D_t(i)$ to be proportional to

$$\frac{1}{1 + \exp(y_i f_{t-1}(x_i))}.$$

A very similar algorithm is described by Duffy and Helmbold [23]. Note that in each case, the weight on the examples, viewed as a vector, is proportional to the negative gradient of the respective loss function. This is because both algorithms are doing a kind of functional gradient descent, an observation that is spelled out and exploited by Breiman [9], Duffy and Helmbold [23], Mason et al. [51, 52] and Friedman [35].

Besides logistic regression, there have been a number of approaches taken to apply boosting to more general regression problems in which the labels y_i are real numbers and the goal is to produce real-valued predictions that are close to these labels. Some of these, such as those of Ridgeway [63] and Freund and Schapire [32], attempt to reduce the regression problem to a classification problem. Others, such as those of Friedman [35] and Duffy and Helmbold [24] use the functional gradient descent view of boosting to derive algorithms that directly minimize a loss function appropriate for regression. Another boosting-based approach to regression was proposed by Drucker [20].

7 Multiclass classification

There are several methods of extending AdaBoost to the multiclass case. The most straightforward generalization [32], called AdaBoost.M1, is adequate when the base learner is strong enough to achieve reasonably high accuracy, even on the hard distributions created by AdaBoost. However, this method fails if the base learner cannot achieve at least 50% accuracy when run on these hard distributions.

For the latter case, several more sophisticated methods have been developed. These generally work by reducing the multiclass problem to a larger binary problem. Schapire and Singer’s [70] algorithm AdaBoost.MH works by creating a set of binary problems, for each example x and each possible label y , of the form: “For example x , is the correct label y or is it one of the other labels?” Freund and Schapire’s [32] algorithm AdaBoost.M2 (which is a special case of Schapire and Singer’s [70] AdaBoost.MR algorithm) instead creates binary problems, for each example x with correct label y and each *incorrect* label y' of the form: “For example x , is the correct label y or y' ?”

These methods require additional effort in the design of the base learning algorithm. A different technique [67], which incorporates Dietterich and Bakiri’s [19] method of error-correcting output codes, achieves similar provable bounds to those of AdaBoost.MH and AdaBoost.M2, but can be used with any base learner that can handle simple, binary labeled data. Schapire and Singer [70] and Allwein, Schapire and Singer [2] give yet another method of combining boosting with error-correcting output codes.

8 Incorporating human knowledge

Boosting, like many machine-learning methods, is entirely data-driven in the sense that the classifier it generates is derived exclusively from the evidence present in the training data itself. When data is abundant, this approach makes sense. However, in some applications, data may be severely limited, but there may be human knowledge that, in principle, might compensate for the lack of data.

In its standard form, boosting does not allow for the direct incorporation of such prior knowledge. Nevertheless, Rochery et al. [64, 65] describe a modification of boosting that combines and balances human expertise with available training data. The aim of the approach is to allow the human’s rough judgments to be refined, reinforced and adjusted by the statistics of the training data, but in a manner that does not permit the data to entirely overwhelm human judgments.

The first step in this approach is for a human expert to construct by hand a rule p mapping each instance x to an estimated probability $p(x) \in [0, 1]$ that is interpreted as the guessed probability that instance x will appear with label $+1$. There are various methods for constructing such a function p , and the hope is that this difficult-to-build function need not be highly accurate for the approach to be effective.

Rochery et al.’s basic idea is to replace the logistic loss function in Eq. (10)

with one that incorporates prior knowledge, namely,

$$\sum_i \ln \left(1 + e^{-y_i f(x_i)} \right) + \eta \sum_i \text{RE} \left(p(x_i) \parallel \frac{1}{1 + e^{-f(x_i)}} \right)$$

where $\text{RE}(p \parallel q) = p \ln(p/q) + (1 - p) \ln((1 - p)/(1 - q))$ is binary relative entropy. The first term is the same as that in Eq. (10). The second term gives a measure of the distance from the model built by boosting to the human’s model. Thus, we balance the conditional likelihood of the data against the distance from our model to the human’s model. The relative importance of the two terms is controlled by the parameter η .

9 Experiments and applications

Practically, AdaBoost has many advantages. It is fast, simple and easy to program. It has no parameters to tune (except for the number of round T). It requires no prior knowledge about the base learner and so can be flexibly combined with *any* method for finding base classifiers. Finally, it comes with a set of theoretical guarantees given sufficient data and a base learner that can reliably provide only moderately accurate base classifiers. This is a shift in mind set for the learning-system designer: instead of trying to design a learning algorithm that is accurate over the entire space, we can instead focus on finding base learning algorithms that only need to be better than random.

On the other hand, some caveats are certainly in order. The actual performance of boosting on a particular problem is clearly dependent on the data and the base learner. Consistent with theory, boosting can fail to perform well given insufficient data, overly complex base classifiers or base classifiers that are too weak. Boosting seems to be especially susceptible to noise [18] (more on this in Sectionsec:exps).

AdaBoost has been tested empirically by many researchers, including [4, 18, 21, 40, 49, 59, 73]. For instance, Freund and Schapire [30] tested AdaBoost on a set of UCI benchmark datasets [54] using C4.5 [60] as a base learning algorithm, as well as an algorithm that finds the best “decision stump” or single-test decision tree. Some of the results of these experiments are shown in Fig. 3. As can be seen from this figure, even boosting the weak decision stumps can usually give as good results as C4.5, while boosting C4.5 generally gives the decision-tree algorithm a significant improvement in performance.

In another set of experiments, Schapire and Singer [71] used boosting for text categorization tasks. For this work, base classifiers were used that test on the presence or absence of a word or phrase. Some results of these experiments comparing